Dynamic Response 2

MATLAB simulations, response plots and brief analysis

Sistemas Dinámicos y Control · Universidad Nacional de Colombia

Source: Juan David Bastidas Rodríguez, 4 Dynamic Response 2.pdf, 39 slides. The presentation cites Franklin, Powell and Emami-Naeini, Feedback Control of Dynamic Systems, 8th ed., Pearson, 2019.

Scope

Every distinct worked example is included: partial fractions; two final-value examples; DC-gain design; three differential equations; cruise control; both DC-motor outputs; and both satellite pulse experiments. Repeated solution slides are combined. The general Laplace tables and symbolic definitions are reference material, rather than additional parameterized systems.

Report case	Presentation slides	Response
1–4	9–17	Partial fractions, final values, DC gain
5–7	18–24	Initial-state and forced ODE solutions
8	29	Car position and speed
9	30–34	Motor speed and angle
10–11	35–39	Single and opposing satellite pulses

Simulation conventions

Time is in seconds. Abstract examples use normalized input/output units. All physical models start from rest; only cases 5–6 use selected nonzero initial conditions, $y(0)=1$ and $y'(0)=0.5$. The 500 N car step and 1 V motor step are explicit simulation choices. The satellite pulse amplitudes and times come directly from the slides.

MATLAB Control System Toolbox `step`, `initial` and `lsim` compute continuous-time LTI responses on the shown uniform time grids. Step/pulse inputs use zero-order hold (ZOH); the smooth exponential input uses first-order hold (FOH). There is no adaptive ODE or Simulink solver setting in these simulations. Black points check the analytical solution or, for the motor, an independently formulated physical state model.

Run the simulations

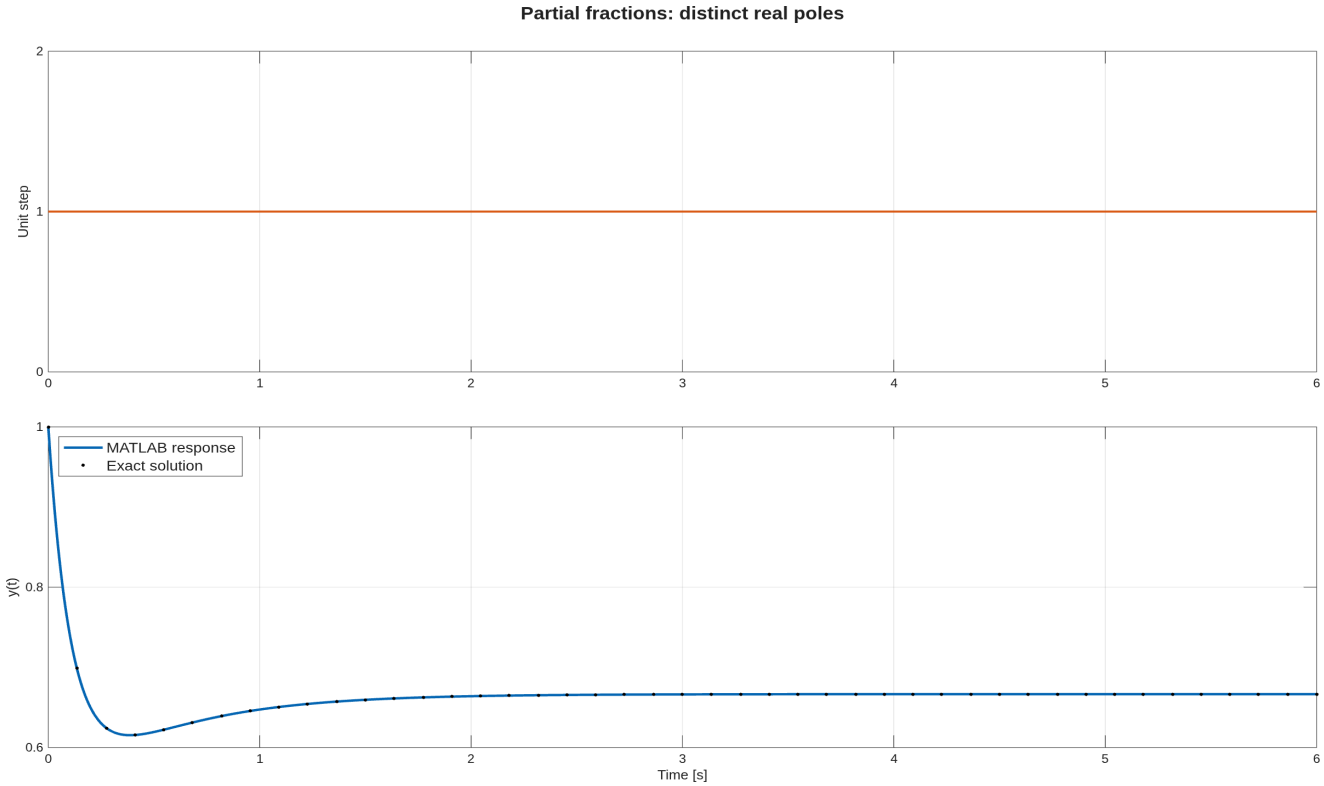
```
% In MATLAB, from this report's folder:
results = run_dynamic_response_2();
% results.outputDir contains PNG plots, MAT data and validation.json
```

The following code excerpts are the executed simulation sections. They share `s = tf('s');` and an output directory. The plotting helper appears at the end; the accompanying .m file is fully runnable.

1 · Partial fractions

Slides 9–10

Y(s)=(s+4)(s+3)/[s(s+9)(s+2)]. To simulate the inverse transform with a unit-step input, use G(s)=sY(s); Y(s) itself is the output transform.



MATLAB code

```
t = (0:0.005:6)'; u = ones(size(t));
% Y(s) is a signal transform; G(s)=s*Y(s) reproduces it with a step.
G = (s+4)*(s+3)/((s+9)*(s+2));
y = step(G,t);
yExact = 2/3 + (10/21)*exp(-9*t) - (1/7)*exp(-2*t);
responsePlot(outputDir,'01',t,u,y,'Unit step','y(t)', ...
    'Partial fractions: distinct real poles',yExact);
```

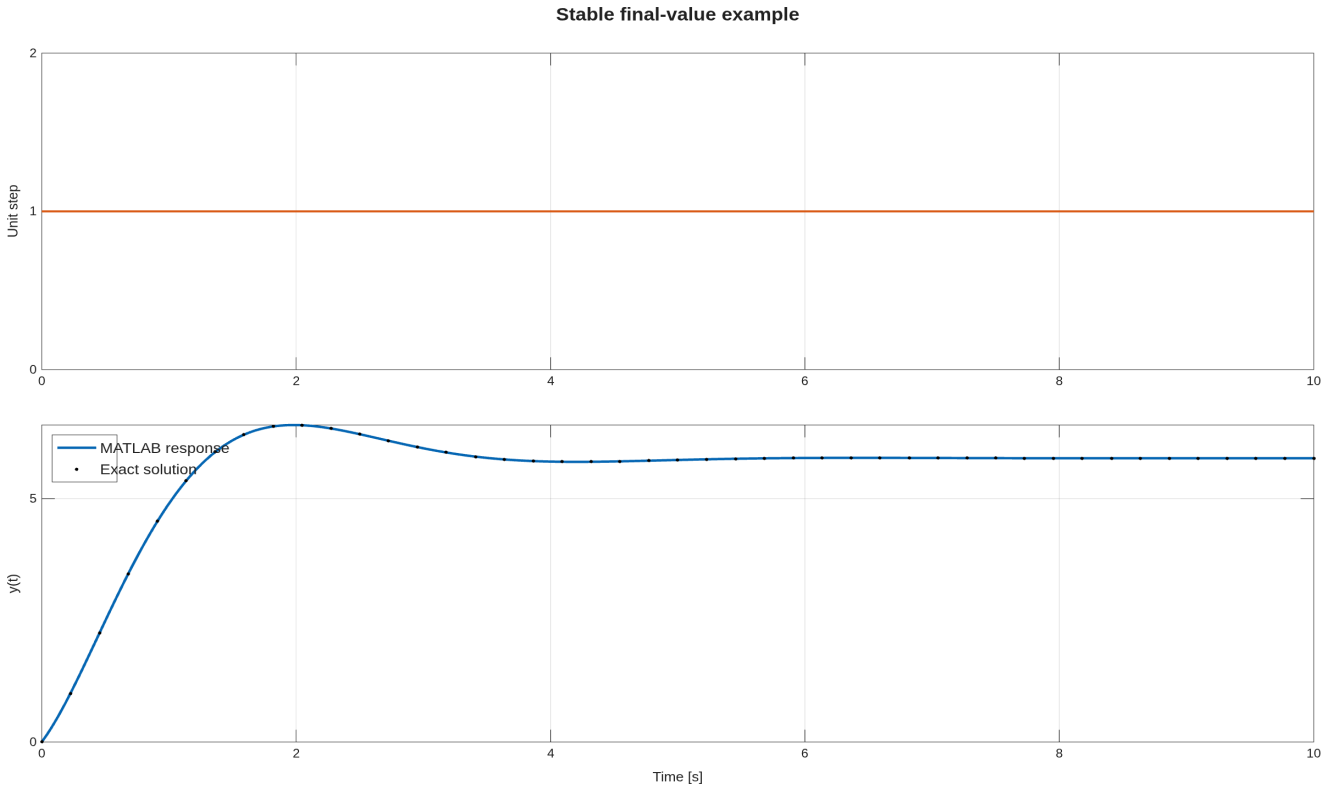
Brief analysis

The response starts at y(0+)=1, falls below its final value, then tends to 2/3. The fast term exp(−9t) decays first; the slower negative exp(−2t) term controls the final approach. The instantaneous jump is consistent with the direct feedthrough of G(s).

2 · A valid final-value calculation

Slides 12–14, example 1

Y(s)=7(s+5)/[2s(s²+2s+3)]. Equivalent step-driven system: G(s)=7(s+5)/[2(s²+2s+3)], with zero initial state.



MATLAB code

```
t = (0:0.005:10)'; u = ones(size(t));
G = 7*(s+5)/(2*(s^2+2*s+3));
y = step(G,t);
w = sqrt(2);
yExact = 35/6-exp(-t).*((35/6)*cos(w*t) ...
    +(7/(3*w))*sin(w*t));
responsePlot(outputDir,'02',t,u,y,'Unit step','y(t)', ...
    'Stable final-value example',yExact);
```

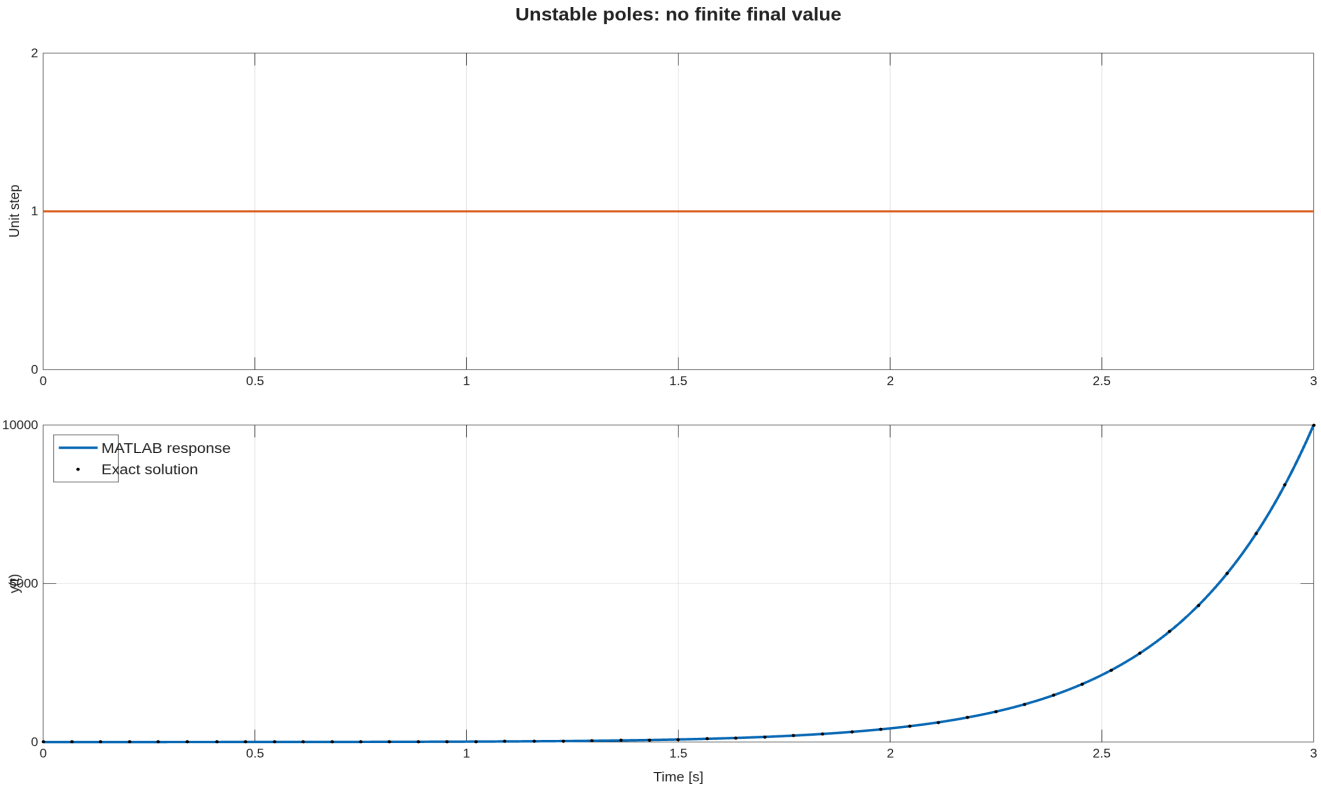
Brief analysis

The system poles are −1 ± j√2. The decaying oscillation approaches 35/6 = 5.833333. The final-value theorem applies because every pole of sY(s) lies strictly in the left half-plane.

3 · An invalid final-value calculation

Slides 13–14, example 2

Y(s)=4/[s(s²−5s+6)]. Equivalent step-driven system: G(s)=4/[(s−2)(s−3)], with zero initial state.



MATLAB code

```
t = (0:0.002:3)'; u = ones(size(t));
G = 4/(s^2-5*s+6);
y = step(G,t);
yExact = 2/3-2*exp(2*t)+(4/3)*exp(3*t);
responsePlot(outputDir,'03',t,u,y,'Unit step','y(t)', ...
    'Unstable poles: no finite final value',yExact);
```

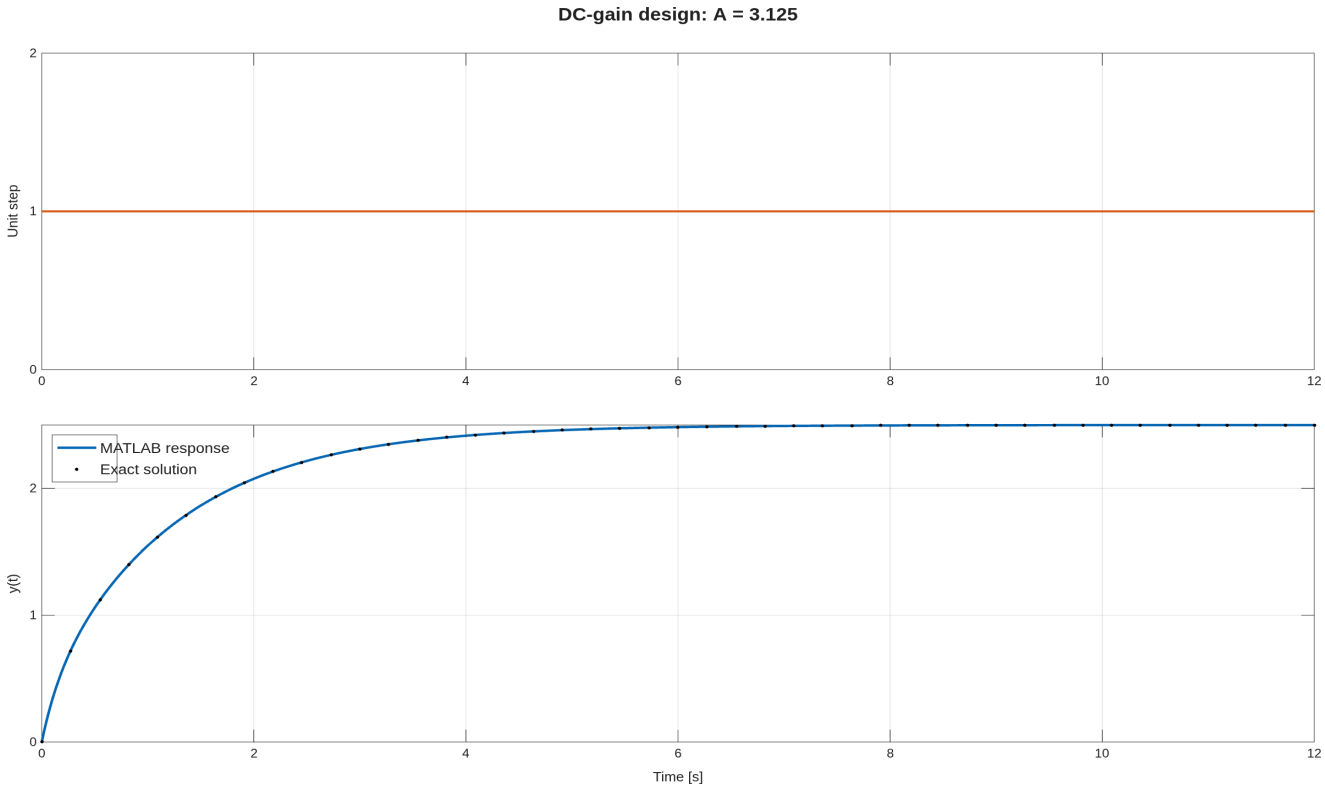
Brief analysis

The output grows to 9997.921 by t=3 s and is unbounded. The poles +2 and +3 generate growing exponentials. Although lim sY(s) as s→0 equals 2/3, this is not a time-domain final value: the theorem's stability condition fails.

4 · Design for a DC gain of 2.5

Slides 15–17

G(s)=4(s+A)/(s²+7s+5). Setting G(0)=4A/5=2.5 gives A=3.125. Apply a unit step with zero initial state.



MATLAB code

```
t = (0:0.01:12)'; u = ones(size(t));
A = 3.125;
G = 4*(s+A)/(s^2+7*s+5);
y = step(G,t);
p = roots([1 7 5]);
c = [1 1; p(1) p(2)]\[-2.5;4];
yExact = 2.5+c(1)*exp(p(1)*t)+c(2)*exp(p(2)*t);
responsePlot(outputDir,'04',t,u,y,'Unit step','y(t)', ...
    'DC-gain design: A = 3.125',yExact);
```

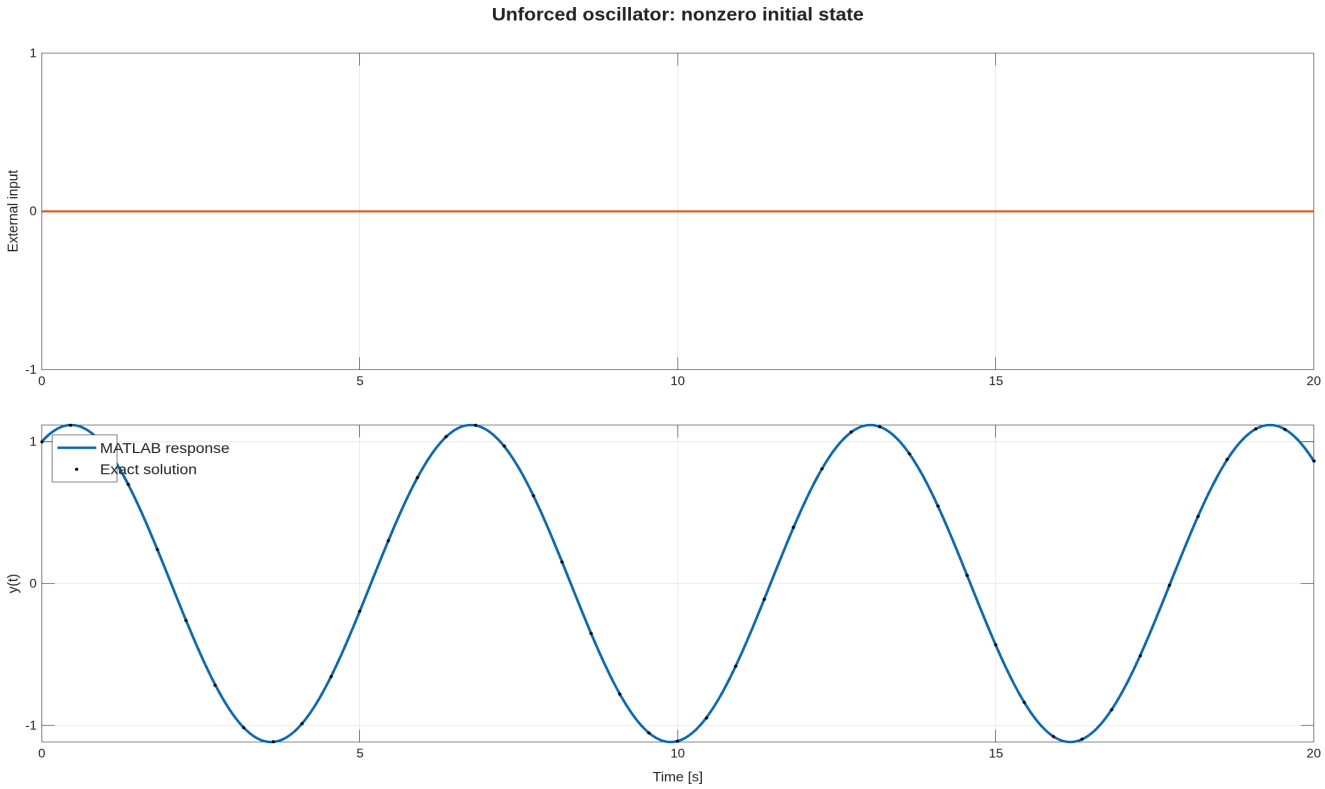
Brief analysis

The output tends monotonically to 2.5. Both poles, approximately −0.8074 and −6.1926, are stable real numbers. The pole closest to zero dominates the slower final approach; the zero at −3.125 changes the transient coefficients.

5 · Unforced oscillator

Slides 18–19

$y''+y=0$; $y(0)=\alpha$, $y'(0)=\beta$. The slides leave α and β symbolic. This run selects $\alpha=1$ and $\beta=0.5$; the input is zero.



MATLAB code

```
t = (0:0.01:20)'; u = zeros(size(t));
alpha = 1; beta = 0.5; % Selected example: y(0), ydot(0).
sys = ss([0 1;-1 0],[0;1],[1 0],0);
y = initial(sys,[alpha;beta],t);
yExact = alpha*cos(t)+beta*sin(t);
responsePlot(outputDir,'05',t,u,y,'External input','y(t)', ...
    'Unforced oscillator: nonzero initial state',yExact);
```

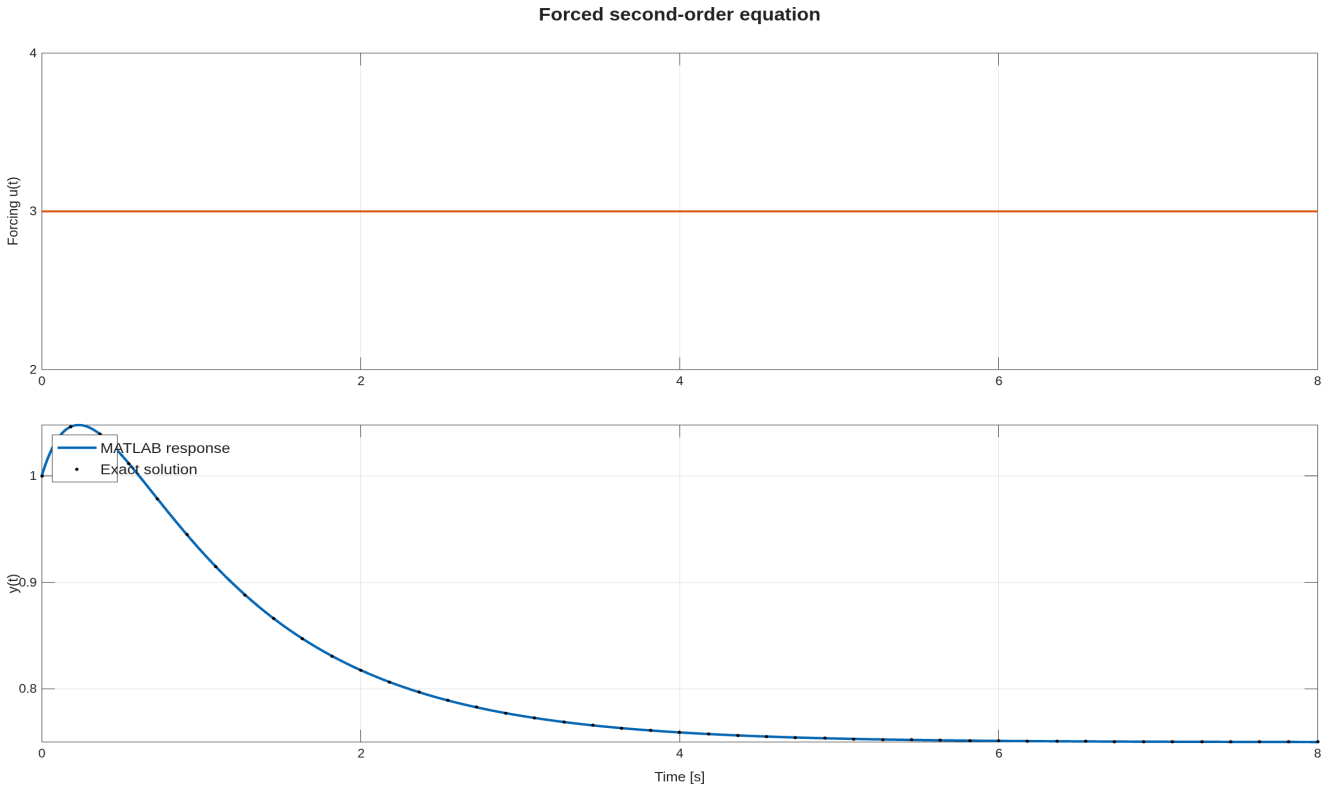
Brief analysis

$y(t)=\cos(t)+0.5\sin(t)$ oscillates indefinitely, with amplitude $\sqrt{1.25}\approx1.1180$ and period $2\pi\approx6.2832$ s. The poles $\pm j$ lie on the imaginary axis, so there is no decay or constant final value. The response comes entirely from stored initial energy.

6 · Forced second-order equation

Slides 20–21

y″+5y′+4y=3. Use state x=[y; y′], input u=3, and selected initial conditions y(0)=1, y′(0)=0.5.



MATLAB code

```
t = (0:0.005:8)'; u = 3*ones(size(t));
alpha = 1; beta = 0.5; % Same selected initial conditions.
sys = ss([0 1;-4 -5],[0;1],[1 0],0);
y = lsim(sys,u,t,[alpha;beta],'zoh');
yExact = 3/4+(-3+beta+4*alpha)/3*exp(-t) ...
    +(3-4*alpha-4*beta)/12*exp(-4*t);
responsePlot(outputDir,'06',t,u,y,'Forcing u(t)','y(t)', ...
    'Forced second-order equation',yExact);
```

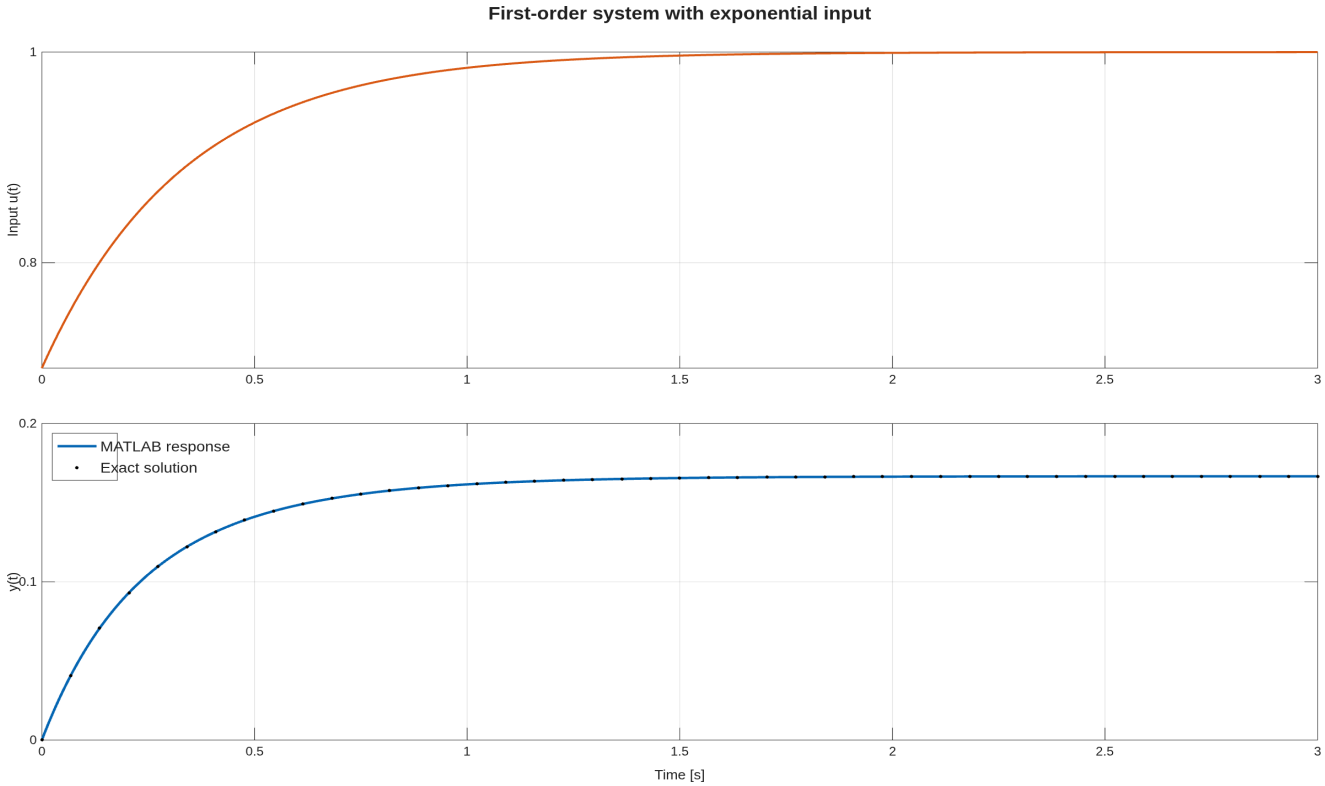
Brief analysis

With these initial values, y(t)=3/4+(1/2)exp(−t)−(1/4)exp(−4t). The response initially rises because y′(0)>0, then decays toward 3/4. The initial conditions change the transient but not the final value because both natural modes are stable.

7 · First order with a varying input

Slides 22–24

y′+6y=u(t), y(0)=0, u(t)=1−0.3 exp(−3t); G(s)=1/(s+6). The smooth input is sampled every 0.001 s and interpolated linearly.



MATLAB code

```
t = (0:0.001:3)'; u = 1-0.3*exp(-3*t);
G = 1/(s+6);
y = lsim(G,u,t,0,'foh');
yExact = 1/6-exp(-3*t)/10-exp(-6*t)/15;
responsePlot(outputDir,'07',t,u,y,'Input u(t)','y(t)', ...
    'First-order system with exponential input',yExact);
```

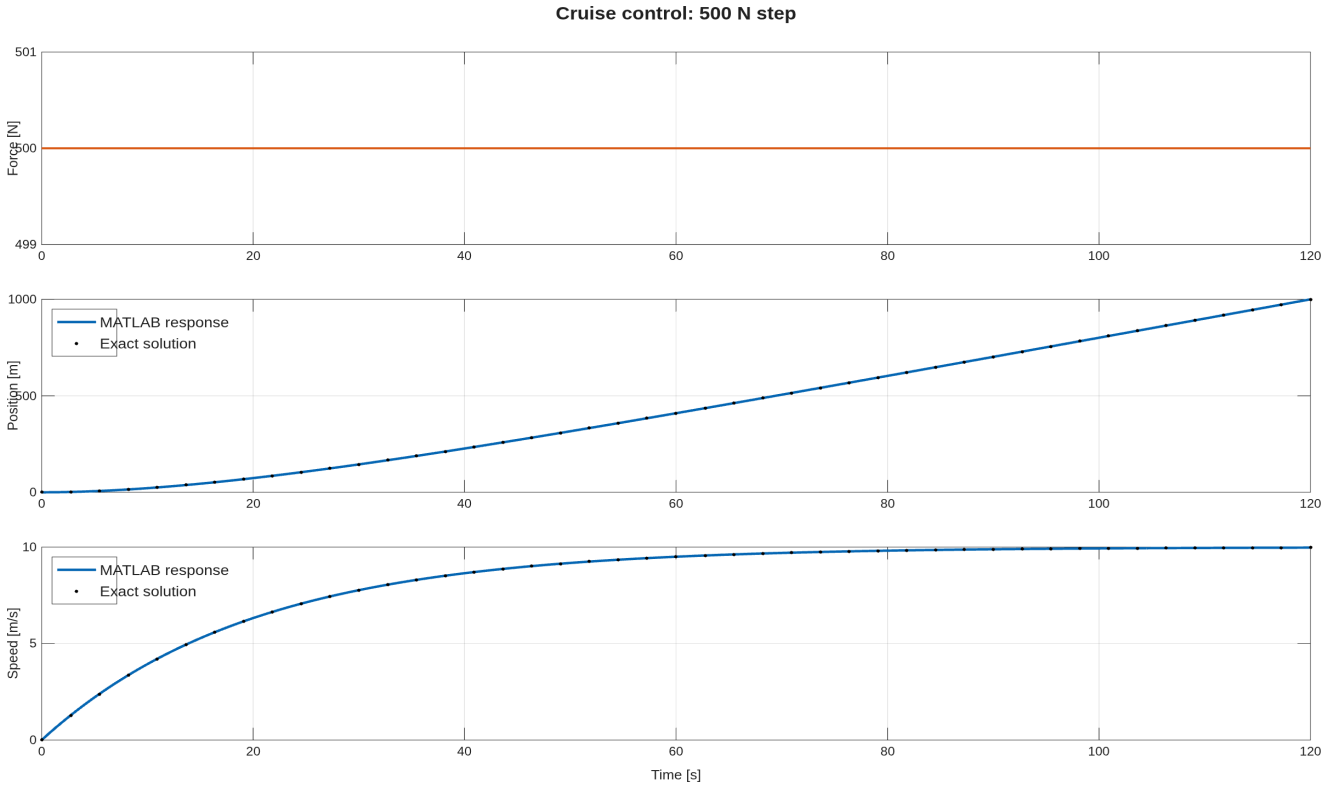
Brief analysis

The input rises from 0.7 to 1 and the output tends to 1/6≈0.166667. The response includes the plant mode exp(−6t) and the slower input mode exp(−3t). The partial-fraction line on slide 22 has sign typos: the residues are +1/6, −1/10 and −1/15. Its final time-domain expression, used here, has the correct negative signs.

8 · Cruise control: position and speed

Slide 29

m x″+b x′=u, with m=1000 kg and b=50 N·s/m, as implied by the slide's Gx(s)=0.001/[s(s+0.05)]. Add Gv(s)=0.001/(s+0.05) to show speed. Apply 500 N from rest.



MATLAB code

```
t = (0:0.05:120)'; u = 500*ones(size(t));
m = 1000; b = 50; % kg, N*s/m; slide 29 transfer coefficients.
Gx = 1/(m*s^2+b*s); Gv = 1/(m*s+b);
x = lsim(Gx,u,t,[],'zoh');
v = lsim(Gv,u,t,[],'zoh');
xExact = 10*(t-20*(1-exp(-t/20)));
vExact = 10*(1-exp(-t/20));
responsePlot(outputDir,'08',t,u,[x v],'Force [N]', ...
    {'Position [m]','Speed [m/s]'},'Cruise control: 500 N step', ...
    [xExact vExact]);
```

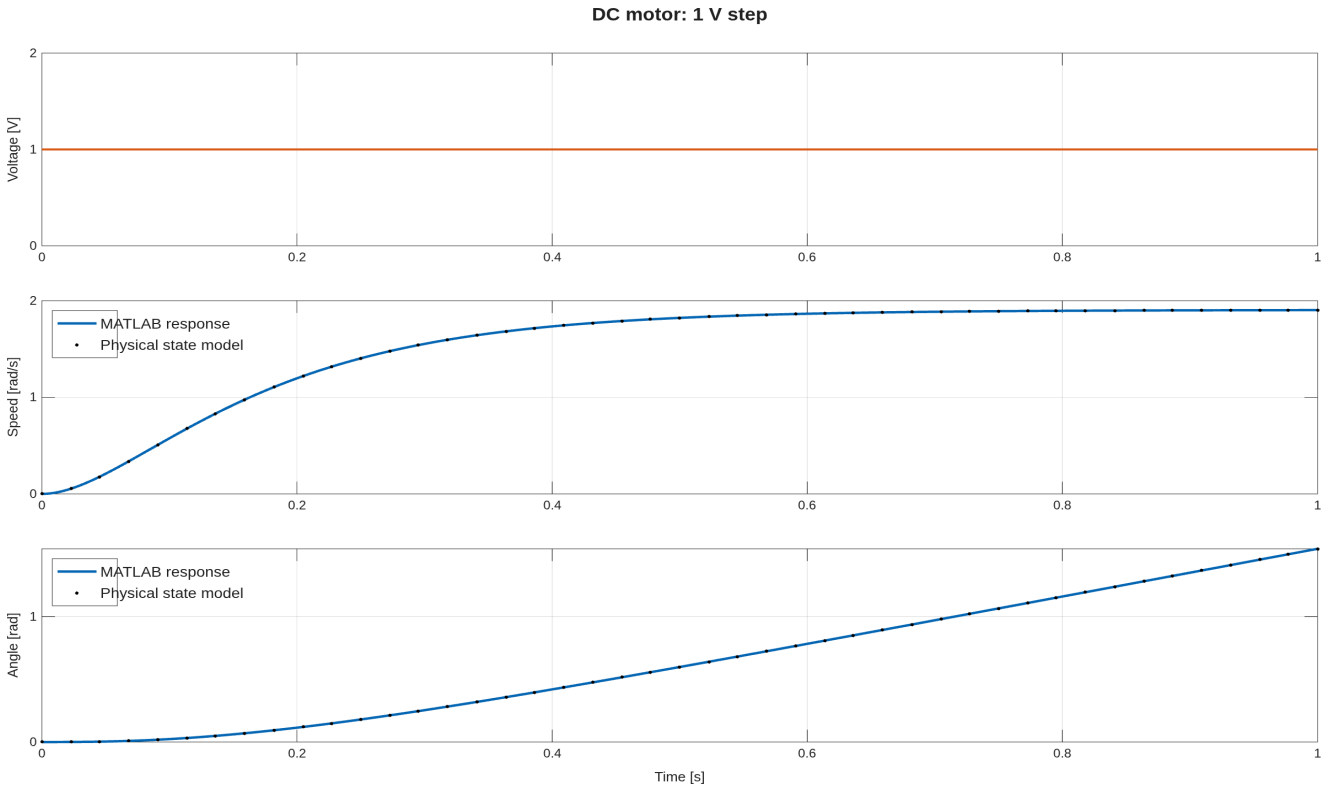
Brief analysis

Speed approaches 10 m/s with time constant 20 s; at 120 s it is 9.975212 m/s. Position is 1000.495750 m and continues growing, approaching a ramp of slope 10 m/s. The position integrator explains why position does not settle even though speed does.

9 · DC motor: speed and angle

Slides 30–34

Jm=0.02 kg·m², b=0.005 N·m·s/rad, Kt=0.5 N·m/A, Ke=0.5 V·s/rad, Ra=2.5 Ω, La=0.1 H. Gω(s)=250/(s²+25.25s+131.25); Gθ(s)=Gω(s)/s. Apply 1 V with zero current, speed and angle; no external load torque.



MATLAB code

```
t = (0:0.001:1)'; u = ones(size(t)); % Armature voltage: 1 V.
Jm = 0.02; b = 0.005; Kt = 0.5; Ke = 0.5;
Ra = 2.5; La = 0.1;
Gw = Kt/((Jm*s+b)*(La*s+Ra)+Kt*Ke);
Gtheta = Gw/s;
omega = step(Gw,t); theta = step(Gtheta,t);
% Independent state model: states [armature current; speed; angle].
Am = [-Ra/La -Ke/La 0; Kt/Jm -b/Jm 0; 0 1 0];
motor = ss(Am,[1/La;0;0],[0 1 0;0 0 1],[0;0]);
ym = lsim(motor,u,t,zeros(3,1),'zoh');
responsePlot(outputDir,'09',t,u,[omega theta],'Voltage [V]', ...
    {'Speed [rad/s]','Angle [rad]'},'DC motor: 1 V step',ym);
```

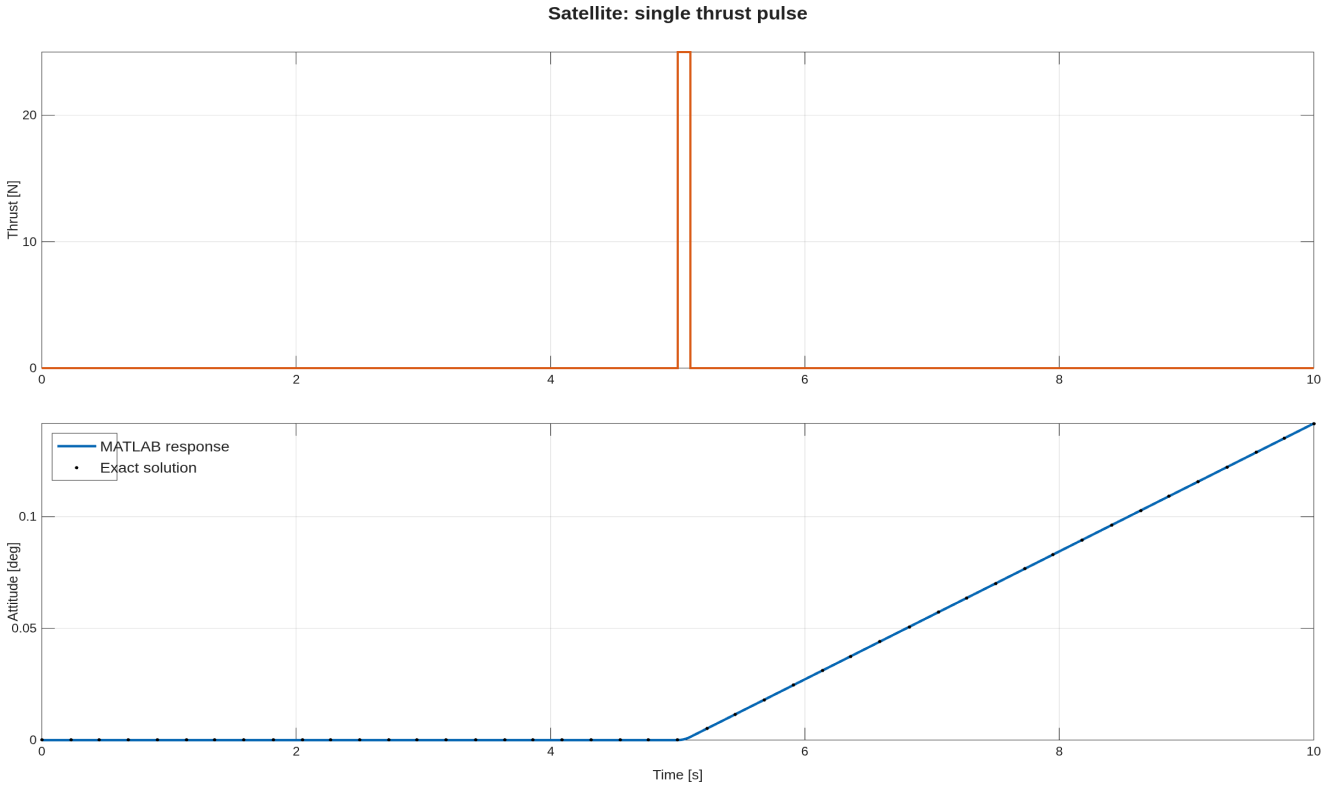
Brief analysis

The speed poles are -17.9298 and -7.3202 s^{-1}. Speed rises without overshoot toward 1.904762 rad/s, reproducing the final motor plot. Angle grows continuously and eventually becomes a ramp with that slope. The additional pole at zero belongs to the angle output, not the speed output.

10 · Satellite: one thrust pulse

Slides 35–37

I θ″=d Fc, with d=1 m, I=5000 kg·m² and zero disturbance torque. H(s)=0.0002/s². Apply +25 N on [5.0,5.1) s; θ(0)=θ′(0)=0.



MATLAB code

```
t = (0:0.01:10)'; k = (0:1000)';
d = 1; I = 5000; H = d/(I*s^2);
u = 25*double(k>=500 & k<510); % Exactly [5.0, 5.1) seconds.
theta = lsim(H,u,t,[],'zoh');
thetaExact = 0.0025*(max(t-5,0).^2-max(t-5.1,0).^2);
responsePlot(outputDir,'10',t,u,rad2deg(theta),'Thrust [N]', ...
    'Attitude [deg]','Satellite: single thrust pulse', ...
    rad2deg(thetaExact));
```

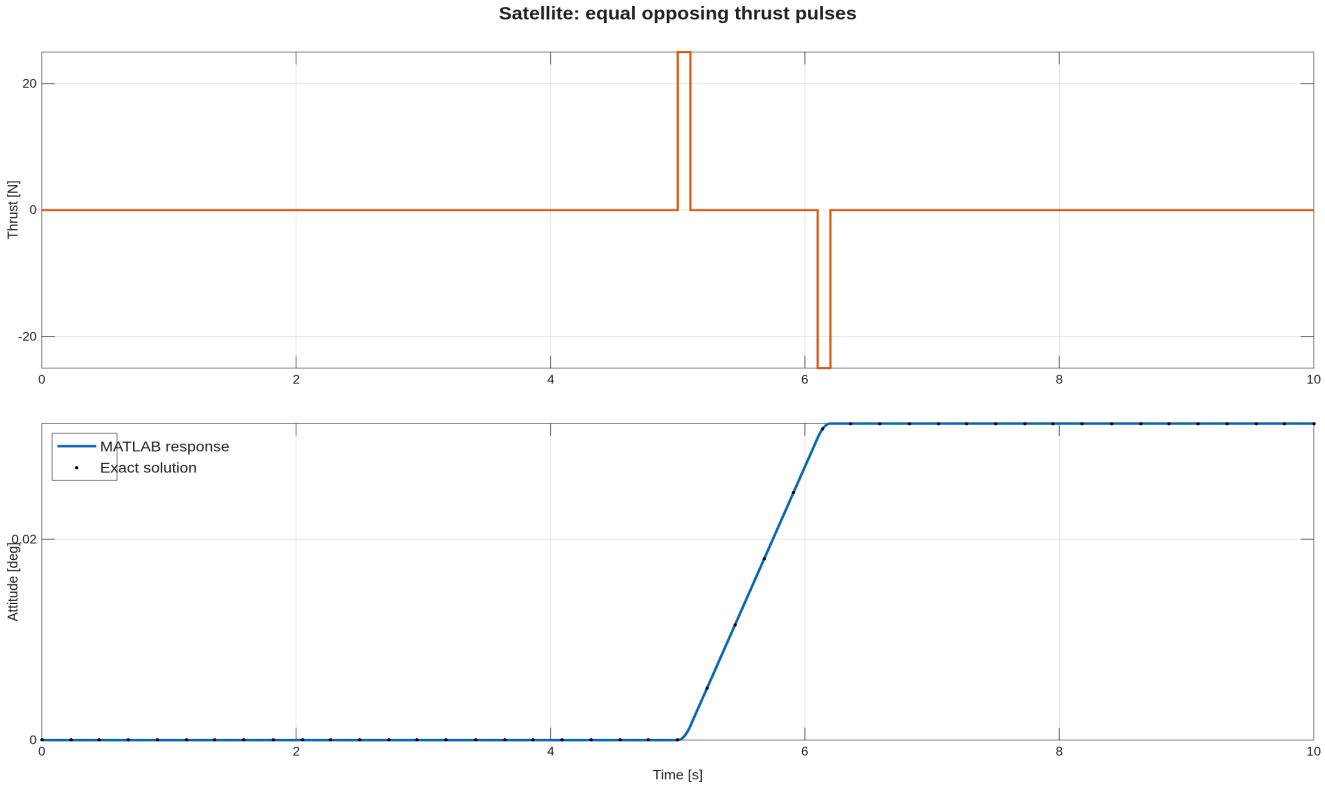
Brief analysis

During thrust the acceleration is 0.005 rad/s². The 2.5 N·s impulse leaves a constant angular speed of 0.0005 rad/s (0.028648 deg/s), so the satellite keeps rotating after the force stops. At 10 s, θ=0.141807°. This explains the ramp at the end of the presentation's single-pulse plot.

11 · Satellite: opposing thrust pulses

Slides 38–39

Use the same satellite and +25 N pulse, then −25 N on [6.1,6.2) s. The grid is 0.01 s and ZOH holds each pulse sample over its full interval.



MATLAB code

```
u = 25*double(k>=500 & k<510) ...
    -25*double(k>=610 & k<620); % Negative pulse [6.1, 6.2).
theta = lsim(H,u,t,[],'zoh');
thetaExact = 0.0025*(max(t-5,0).^2-max(t-5.1,0).^2 ...
    -max(t-6.1,0).^2+max(t-6.2,0).^2);
omega = lsim(d/(I*s),u,t,[],'zoh');
responsePlot(outputDir,'11',t,u,rad2deg(theta),'Thrust [N]', ...
    'Attitude [deg]','Satellite: equal opposing thrust pulses', ...
    rad2deg(thetaExact));
```

Brief analysis

The second pulse removes the angular speed created by the first, so the attitude remains constant after 6.2 s. Its final value is 0.000550 rad = 0.031513°. Equal opposing impulses cancel velocity, not the displacement accumulated between them. The flat ending matches the presentation. ZOH and explicit time masks avoid the slide code's one-sample timing shift and edge interpolation.

Numerical validation and reproducibility

Executed with MATLAB 26.1.0.3346908 (R2026a) Update 5 and Control System Toolbox. All 19 numerical assertions passed. The graphs in this PDF are MATLAB exports, not reconstructed illustrations.

Check	Maximum absolute error	Tolerance
Partial fractions	5.55e-16	1e-10
Stable final value response	1.24e-14	1e-10
Stable DC gain	0	1e-12
Unstable response	1.99e-09	1e-06
DC-gain response	8.88e-15	1e-10
DC gain target	0	1e-12
Homogeneous ODE	3.31e-14	1e-10
Forced ODE	7.44e-15	1e-10
First-order ODE, FOH input	1.88e-08	3e-08
Car position	2.39e-11	1e-07
Car speed	3.59e-13	1e-09
Motor TF versus physical states	2.98e-14	1e-10
Motor steady speed	0	1e-12
Satellite single pulse [rad]	3.6e-17	1e-12
Satellite double pulse [rad]	2.35e-17	1e-12
Satellite final angle [rad]	1.52e-18	1e-12
Satellite residual speed [rad/s]	0	1e-12
Positive pulse impulse [N*s]	0	1e-12
Negative pulse impulse [N*s]	0	1e-12

What was checked

Closed-form time responses check the partial fractions, final-value cases, gain design, three ODEs, cruise control and satellite dynamics. The motor’s transfer-function output is compared with current/speed/angle state equations. Additional checks verify DC gains, thrust impulses, final attitude and zero residual angular speed. The small FOH discrepancy is from approximating the exponential input by straight segments.

Files and rerun instructions

`run_dynamic_response_2.m` contains all simulation and validation code. It creates a fresh folder under `artifacts/dynamic_response_2/` by default. The returned folder contains eleven PNG plots, `simulation_data.mat` and `validation.json`. The MAT file stores radians for satellite angles; the plots convert to degrees.

```
python build_report.py /path/returned/in/results.outputDir
```

The PDF assembly step requires Python with reportlab. Numerical checks validate these ideal linear models; no hardware experiment is claimed. The shared MATLAB session was unavailable, so execution used a separate batch process. Existing user models and workspaces were preserved.

Runtime note: MATLAB reported that graphics acceleration hardware was unavailable. All figure exports completed and the batch process exited normally (status 0). Code Analyzer returned no issues.

Shared MATLAB plotting helper

Every case uses this helper to place the input above its output(s), label physical units, overlay reference samples and export the figure. Only the figure created by this function is closed. The enclosing function sets `s=tf('s')` and creates `outputDir` before running the excerpts.

```
function responsePlot(folder,id,t,u,y,inputLabel,outputLabels,heading,reference)
% Own only this figure; preserve unrelated desktop figures and root defaults.
if ischar(outputLabels), outputLabels = {outputLabels}; end
fig = figure('Visible','off','Color','w','Position',[50 50 1100 650]);
cleanupFigure = onCleanup(@() close(fig));
tl = tiledlayout(fig,size(y,2)+1,1,'TileSpacing','compact','Padding','compact');
title(tl,heading,'FontSize',15,'FontWeight','bold');
ax = nexttile(tl);
set(ax,'FontSize',15);
if strcmp(id,'07')
    plot(ax,t,u,'Color',[0.85 0.35 0.08],'LineWidth',1.7);
else
    stairs(ax,t,u,'Color',[0.85 0.35 0.08],'LineWidth',1.7);
end
ylabel(ax,inputLabel); grid(ax,'on'); xlim(ax,[t(1) t(end)]);
for j = 1:size(y,2)
    ax = nexttile(tl);
    set(ax,'FontSize',15);
    plot(ax,t,y(:,j),'Color',[0.02 0.4 0.7],'LineWidth',2);
    hold(ax,'on');
    indices = unique(round(linspace(1,numel(t),45)));
    plot(ax,t(indices),reference(indices,j),'k.','MarkerSize',8);
    ylabel(ax,outputLabels{j}); grid(ax,'on'); xlim(ax,[t(1) t(end)]);
    if strcmp(id,'09'), referenceName = 'Physical state model';
    else, referenceName = 'Exact solution'; end
    legend(ax,'MATLAB response',referenceName, ...
        'Location','northwest','FontSize',12);
end
xlabel(ax,'Time [s]');
drawnow;
exportgraphics(fig,fullfile(folder,[id '.png']),'Resolution',170);
end
```

Case 11 continues case 10 and reuses its time grid, satellite model and pulse sample indices. Running the complete .m function executes the cases in order and performs the checks automatically.